

{ JS }

JavaScript desde Cero

La guía definitiva para aprender el lenguaje más popular del mundo paso a paso

Por Carlos Mendoza

Edición 2025 · 40 páginas · Nivel principiante a intermedio

JavaScript desde Cero

Autor: Carlos Mendoza

Edición: 1.0 — 2025

Idioma: Español

Formato: PDF — eBook

Todos los derechos reservados. Queda prohibida la reproducción total o parcial de este material sin la autorización expresa del autor.

Este libro es una guía educativa creada para ayudar a personas sin experiencia previa a aprender JavaScript de manera práctica y progresiva.

Tabla de Contenido

01	Bienvenida al Mundo de JavaScript Qué es, historia y por qué aprenderlo
02	Configurando tu Entorno Herramientas, editores y primer archivo
03	Variables y Tipos de Datos var, let, const y los 8 tipos de JS
04	Operadores Aritméticos, lógicos, comparación y más
05	Estructuras de Control if, else, switch y el flujo del programa
06	Bucles y Repetición for, while, do-while y forEach
07	Funciones Declaración, expresión, arrow functions
08	Arrays Listas, métodos y manipulación de datos

09	Objetos Propiedades, métodos y JSON
10	El DOM Manipula HTML con JavaScript
11	Eventos Clicks, teclado, formularios y más
12	Próximos Pasos Qué aprender después de este libro

CAPÍTULO 01

Bienvenida al Mundo de JavaScript

Descubre por qué JS es el lenguaje más importante de la web y cómo cambiará tu vida profesional.

¿Qué es JavaScript?

JavaScript es un lenguaje de programación dinámico, interpretado y de alto nivel. Fue creado en **1995** por Brendan Eich en tan solo 10 días mientras trabajaba en Netscape. A pesar de su origen apresurado, se convirtió en el lenguaje estándar de la web y hoy está presente en prácticamente todos los sitios y aplicaciones del mundo.

A diferencia de HTML (que estructura) y CSS (que da estilo), JavaScript **da vida** a las páginas web: menús desplegables, animaciones, formularios interactivos, juegos en el navegador y mucho más.

¿Por qué aprender JavaScript?

- Es el único lenguaje que corre nativamente en todos los navegadores.
- Es el lenguaje #1 en demanda laboral desde hace más de 10 años.
- Con React Native puedes crear apps móviles con el mismo lenguaje.
- Con Node.js puedes programar servidores backend también.
- Es gratuito, open source y tiene la mayor comunidad de devs del mundo.

■ **Tip:** No necesitas instalar nada para empezar. Abre tu navegador, presiona F12 y tendrás una consola JavaScript lista para usar ahora mismo.

Historia resumida

Año	Hito importante
1995	Brendan Eich crea JavaScript en 10 días
1997	Se estandariza como ECMAScript (ES1)
2009	Node.js lleva JS al servidor
2015	ES6 moderniza el lenguaje radicalmente

2023	JS es el lenguaje más popular del mundo (Stack Overflow)
------	--

CAPÍTULO 02

Configurando tu Entorno

Antes de escribir código necesitas las herramientas correctas. Vamos paso a paso.

El editor de código: VS Code

Visual Studio Code (VS Code) es el editor más popular entre desarrolladores web. Es gratuito, ligero, tiene miles de extensiones y soporte nativo para JavaScript. Descárgalo en code.visualstudio.com

Extensiones recomendadas para JS:

- Prettier – formatea tu código automáticamente
- ESLint – detecta errores mientras escribes
- JavaScript (ES6) code snippets – atajos de código
- Live Server – recarga el navegador automáticamente
- GitLens – integración con Git

Tu primer archivo JavaScript

Crea una carpeta llamada **mis-proyectos-js** y dentro un archivo **index.html** con el siguiente contenido:

```
<!DOCTYPE html>
<html lang='es'>
<head>
  <meta charset='UTF-8'>
  <title>Mi primer JS</title>
</head>
<body>
  <h1>Hola Mundo</h1>
  <script src='app.js'></script>
</body>
```



```
</html>
```

Luego crea **app.js** en la misma carpeta:

```
// Mi primer programa en JavaScript
```

```
console.log('¡Hola, mundo!');
```

```
alert('¡JavaScript funciona!');
```

■ **Tip:** La consola del navegador (F12 → Console) es tu mejor amiga al aprender. Úsala constantemente para probar código y ver mensajes de error.

CAPÍTULO 03

Variables y Tipos de Datos

Las variables son cajitas donde guardamos información. Aprende a declararlas correctamente.

¿Qué es una variable?

Una variable es un espacio en memoria con un nombre al que podemos asignar un valor y modificarlo durante el programa. En JavaScript moderno usamos **let** y **const**.

```
// Con let: el valor puede cambiar
let edad = 25;
edad = 26; // esto es válido

// Con const: el valor NO puede cambiar
const nombre = 'Carlos';
// nombre = 'Juan'; ← esto daría ERROR

// var (forma antigua, evitar en código nuevo)
var ciudad = 'México';
```

Los 8 tipos de datos en JavaScript

Tipo	Ejemplo	Descripción
String	'Hola' / "mundo"	Texto entre comillas
Number	42 / 3.14 / -7	Números enteros y decimales
Boolean	true / false	Verdadero o falso
Null	null	Ausencia intencional de valor
Undefined	undefined	Variable declarada sin valor
Object	{ nombre: 'Ana' }	Colección de propiedades

Array	[1, 2, 3]	Lista ordenada de valores
Symbol	Symbol('id')	Identificador único (avanzado)

typeof: conoce el tipo de un valor

```
console.log(typeof 'Hola');    // 'string'
console.log(typeof 42);        // 'number'
console.log(typeof true);     // 'boolean'
console.log(typeof undefined); // 'undefined'
console.log(typeof null);     // 'object' ← peculiaridad histórica de JS
console.log(typeof {});      // 'object'
console.log(typeof []);      // 'object'
```

■ **Tip:** Usa siempre **const** por defecto y cambia a **let** sólo cuando necesites reasignar el valor. Nunca uses **var** en código moderno.

CAPÍTULO 04

Operadores

Con los operadores podemos hacer cálculos, comparaciones y combinar condiciones.

Operadores Aritméticos

```
let a = 10, b = 3;

console.log(a + b);    // 13 – suma
console.log(a - b);    // 7  – resta
console.log(a * b);    // 30 – multiplicación
console.log(a / b);    // 3.333... – división
console.log(a % b);    // 1  – módulo (resto)
console.log(a ** b);   // 1000 – potencia (ES6)
console.log(++a);      // 11 – incremento
console.log(--b);      // 2  – decremento
```

Operadores de Comparación

```
// == compara valor (con conversión de tipo)
console.log(5 == '5');    // true ← peligroso

// === compara valor Y tipo (estricto)
console.log(5 === '5');   // false ← correcto
console.log(5 === 5);     // true

console.log(10 > 5);      // true
console.log(10 < 5);      // false
console.log(10 >= 10);    // true
console.log(10 !== 9);    // true
```

Operadores Lógicos

```
// && = AND (ambas condiciones deben ser true)
```

```
console.log(true && true); // true
```

```
console.log(true && false); // false
```

```
// || = OR (al menos una debe ser true)
```

```
console.log(false || true); // true
```

```
console.log(false || false); // false
```

```
// ! = NOT (invierte el valor)
```

```
console.log(!true); // false
```

```
console.log(!false); // true
```

■ **Tip:** Siempre usa `===` en lugar de `==` para comparaciones. El operador `==` hace conversiones de tipo automáticas que pueden generar resultados inesperados.

CAPÍTULO 05

Estructuras de Control

Toma decisiones en tu código. Haz que el programa haga cosas diferentes según las condiciones.

if / else if / else

```
const temperatura = 28;

if (temperatura >= 30) {
  console.log('Hace mucho calor');
} else if (temperatura >= 20) {
  console.log('Temperatura agradable');
} else if (temperatura >= 10) {
  console.log('Hace frío');
} else {
  console.log('Hace mucho frío');
}

// Resultado: 'Temperatura agradable'
```

Operador Ternario

Una forma compacta de escribir un if/else simple:

```
const edad = 20;

const acceso = edad >= 18 ? 'Permitido' : 'Denegado';

console.log(acceso); // 'Permitido'
```

switch

```
const dia = 'lunes';
```

```
switch (dia) {  
  case 'lunes':  
  case 'martes':  
  case 'miércoles':  
  case 'jueves':  
  case 'viernes':  
    console.log('Es día laboral');  
    break;  
  case 'sábado':  
  case 'domingo':  
    console.log('Es fin de semana');  
    break;  
  default:  
    console.log('Día no reconocido');  
}
```

■ **Tip:** No olvides el **break** en cada caso del switch. Sin él, el código continúa ejecutando los casos siguientes (fall-through).

CAPÍTULO 06

Bucles y Repetición

Repite acciones sin escribir el mismo código una y otra vez.

Bucle for

```
// for clásico: contar del 1 al 5
for (let i = 1; i <= 5; i++) {
  console.log('Vuelta número: ' + i);
}

// for...of: recorre arrays
const frutas = ['manzana', 'pera', 'uva'];
for (const fruta of frutas) {
  console.log(fruta);
}

// for...in: recorre propiedades de un objeto
const persona = { nombre: 'Ana', edad: 30 };
for (const clave in persona) {
  console.log(clave + ': ' + persona[clave]);
}
```

Bucle while y do-while

```
// while: ejecuta mientras la condición sea true
let contador = 0;
while (contador < 3) {
  console.log('Contador: ' + contador);
}
```



```
    contador++;  
  }  
  
  // do-while: ejecuta AL MENOS una vez  
  
  let num = 10;  
  
  do {  
  
    console.log('Num es: ' + num);  
  
    num++;  
  
  } while (num < 5); // Se ejecutó una vez aunque 10 > 5
```

break y continue

```
for (let i = 0; i < 10; i++) {  
  if (i === 3) continue; // salta el 3  
  
  if (i === 7) break;    // para en el 7  
  
  console.log(i);       // 0,1,2,4,5,6  
  
}
```

■ **Tip:** Evita los bucles infinitos: asegúrate siempre de que la condición eventualmente se vuelva falsa. Un bucle infinito congela el navegador.

CAPÍTULO 07

Funciones

Las funciones son bloques de código reutilizable. Son la base de la programación modular.

Declaración de función (Function Declaration)

```
function saludar(nombre) {  
  return 'Hola, ' + nombre + '!';  
}  
  
console.log(saludar('Carlos')); // 'Hola, Carlos!'  
console.log(saludar('María')); // 'Hola, María!'
```

Función Expresión (Function Expression)

```
const sumar = function(a, b) {  
  return a + b;  
};  
  
console.log(sumar(5, 3)); // 8
```

Arrow Functions (ES6) — la forma moderna

```
// Forma larga  
const multiplicar = (a, b) => {  
  return a * b;  
};  
  
// Forma corta (una expresión, return implícito)  
const duplicar = n => n * 2;
```

```
console.log(multiplicar(4, 5)); // 20
console.log(duplicar(7));      // 14
```

Parámetros por defecto y rest

```
// Parámetro por defecto
function presentar(nombre, pais = 'México') {
  return nombre + ' es de ' + pais;
}

console.log(presentar('Carlos')); // Carlos es de México
console.log(presentar('Ana', 'Colombia')); // Ana es de Colombia

// Parámetro rest (...args)
function sumarTodo(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}

console.log(sumarTodo(1, 2, 3, 4, 5)); // 15
```

■ **Tip:** Nombra tus funciones con verbos descriptivos: **obtenerUsuario()**, **calcularTotal()**, **mostrarMensaje()**. El código legible es código profesional.

CAPÍTULO 08

Arrays

Los arrays son listas ordenadas. Son una de las estructuras más usadas en JavaScript.

Creación y acceso

```
const colores = ['rojo', 'verde', 'azul'];

console.log(colores[0]);      // 'rojo'   (índice desde 0)
console.log(colores[2]);      // 'azul'
console.log(colores.length);  // 3

// Modificar un elemento
colores[1] = 'amarillo';
console.log(colores); // ['rojo', 'amarillo', 'azul']
```

Métodos esenciales

```
const nums = [1, 2, 3, 4, 5];

// Agregar y quitar
nums.push(6);      // agrega al final → [1,2,3,4,5,6]
nums.pop();         // quita el último → [1,2,3,4,5]
nums.unshift(0);    // agrega al inicio → [0,1,2,3,4,5]
nums.shift();       // quita el primero → [1,2,3,4,5]

// Buscar
console.log(nums.indexOf(3)); // 2
console.log(nums.includes(10)); // false

// Transformar
```

```
const dobles = nums.map(n => n * 2); // [2,4,6,8,10]
const pares = nums.filter(n => n % 2 === 0); // [2,4]
const suma = nums.reduce((acc, n) => acc + n, 0); // 15
```

Desestructuración de arrays

```
const [primero, segundo, ...resto] = [10, 20, 30, 40, 50];
console.log(primero); // 10
console.log(segundo); // 20
console.log(resto); // [30, 40, 50]
```

■ **Tip:** `map()`, `filter()` y `reduce()` son los tres métodos más poderosos de los arrays. Dominarlos te llevará al siguiente nivel.

Spread y copia de arrays

```
const original = [1, 2, 3];

// Copia con spread
const copia = [...original];
copia.push(4);
console.log(original); // [1, 2, 3] – sin cambios
console.log(copia); // [1, 2, 3, 4]

// Combinar arrays
const a = [1, 2];
const b = [3, 4];
const combinado = [...a, ...b]; // [1, 2, 3, 4]

// Array.from
const letras = Array.from('Hola'); // ['H','o','l','a']
const rango = Array.from({length: 5}, (_, i) => i + 1); // [1,2,3,4,5]
```

Métodos de búsqueda y ordenamiento

```
const productos = [  
  { nombre: 'Teclado', precio: 450 },  
  { nombre: 'Mouse', precio: 200 },  
  { nombre: 'Monitor', precio: 3500 },  
  { nombre: 'Webcam', precio: 800 },  
];  
  
// find: primer elemento que cumple la condición  
const caro = productos.find(p => p.precio > 700);  
console.log(caro.nombre); // 'Monitor'  
  
// findIndex: índice del primer elemento encontrado  
const idx = productos.findIndex(p => p.nombre === 'Mouse');  
console.log(idx); // 1  
  
// sort: ordenar (MODIFICA el array original)  
const porPrecio = [...productos].sort((a, b) => a.precio - b.precio);  
// [Mouse 200, Teclado 450, Webcam 800, Monitor 3500]  
  
// some y every  
const algoCaro = productos.some(p => p.precio > 1000); // true  
const todosCaro = productos.every(p => p.precio > 100); // true
```

CAPÍTULO 09

Objetos

Los objetos representan entidades del mundo real con propiedades y comportamientos.

Creación y acceso a propiedades

```
const persona = {  
  nombre: 'Carlos',  
  edad: 28,  
  ciudad: 'Guadalajara',  
  activo: true  
};  
  
// Acceso con punto  
console.log(persona.nombre); // 'Carlos'  
  
// Acceso con corchetes  
console.log(persona['edad']); // 28  
  
// Agregar propiedad  
persona.email = 'carlos@email.com';  
  
// Eliminar propiedad  
delete persona.activo;
```

Métodos dentro de objetos

```
const calculadora = {  
  sumar(a, b) { return a + b; },  
  restar(a, b) { return a - b; },  
  PI: 3.14159  
};
```

```
};  
  
console.log(calculadora.sumar(5, 3)); // 8  
console.log(calculadora.PI); // 3.14159
```

Desestructuración de objetos

```
const { nombre, edad, ciudad = 'No especificada' } = persona;  
  
console.log(nombre); // 'Carlos'  
console.log(edad); // 28  
  
// Spread operator  
const copia = { ...persona, pais: 'México' };
```

■ **Tip:** JSON (JavaScript Object Notation) está basado en la sintaxis de objetos JS. Convierte objetos con **JSON.stringify()** y parsea con **JSON.parse()**.

JSON: el formato de intercambio de datos

```
const usuario = {  
  nombre: 'Carlos',  
  edad: 28,  
  habilidades: ['JavaScript', 'HTML', 'CSS']  
};  
  
// Objeto → JSON string  
const json = JSON.stringify(usuario);  
console.log(json);  
// '{"nombre":"Carlos","edad":28,"habilidades":["JavaScript","HTML","CSS"]}'  
  
// JSON string → Objeto  
const objeto = JSON.parse(json);  
console.log(objeto.nombre); // 'Carlos'
```



```
// JSON con formato legible  
console.log(JSON.stringify(usuario, null, 2));
```

Object: métodos útiles

```
const config = { host: 'localhost', port: 3000, debug: true };  
  
// Obtener claves  
console.log(Object.keys(config));  
// ['host', 'port', 'debug']  
  
// Obtener valores  
console.log(Object.values(config));  
// ['localhost', 3000, true]  
  
// Obtener pares clave-valor  
console.log(Object.entries(config));  
// [['host','localhost'], ['port',3000], ['debug',true]]  
  
// Combinar objetos  
const extra = { version: '1.0', timeout: 5000 };  
const completo = Object.assign({}, config, extra);  
// equivalente con spread: const completo = { ...config, ...extra };
```

CAPÍTULO 10

El DOM

Document Object Model: la forma en que JavaScript se comunica con el HTML de la página.

¿Qué es el DOM?

El DOM es una representación en árbol de todos los elementos de tu página HTML. JavaScript puede acceder y modificar cualquier elemento: cambiar textos, colores, mostrar/ocultar secciones, agregar elementos nuevos y mucho más.

Seleccionar elementos

```
// Por ID (devuelve 1 elemento)
const titulo = document.getElementById('mi-titulo');

// Por clase CSS (devuelve colección)
const botones = document.getElementsByClassName('btn');

// Selector CSS moderno (recomendado)
const parrafo = document.querySelector('p');
const todosLosParrafos = document.querySelectorAll('p');
const primero = document.querySelector('#lista li:first-child');
```

Modificar el DOM

```
const titulo = document.querySelector('h1');

// Cambiar texto
titulo.textContent = 'Nuevo título';

// Cambiar HTML interno
titulo.innerHTML = 'Título con <em>énfasis</em>';
```

```
// Cambiar estilos
titulo.style.color = 'blue';
titulo.style.fontSize = '2rem';

// Cambiar clases
titulo.classList.add('activo');
titulo.classList.remove('inactivo');
titulo.classList.toggle('visible');

// Crear elemento nuevo
const nuevoParrafo = document.createElement('p');
nuevoParrafo.textContent = 'Párrafo creado con JS';
document.body.appendChild(nuevoParrafo);
```

■ **Tip:** Usa siempre **querySelector** y **querySelectorAll** — son los métodos modernos y más flexibles para seleccionar elementos del DOM.

CAPÍTULO 11

Eventos

Los eventos hacen que tu página reaccione a las acciones del usuario.

addEventListener: la forma correcta

```
const boton = document.querySelector('#mi-boton');

boton.addEventListener('click', function() {
  alert('¡Hiciste clic!');
});

// Con arrow function
boton.addEventListener('click', () => {
  console.log('Click detectado');
});
```

Eventos más comunes

Evento	Cuándo ocurre
click	Al hacer clic en un elemento
dblclick	Al hacer doble clic
mouseover	Al pasar el mouse por encima
mouseout	Al sacar el mouse del elemento
keydown	Al presionar una tecla
keyup	Al soltar una tecla
submit	Al enviar un formulario
change	Al cambiar el valor de un input

load	Al cargar la página completamente
DOMContentLoaded	Al cargar el HTML (sin esperar imágenes)

El objeto event

```
document.addEventListener('keydown', (event) => {  
  console.log('Tecla presionada:', event.key);  
  if (event.key === 'Enter') {  
    console.log('¡Presionaste Enter!');  
  }  
});
```

```
const form = document.querySelector('form');  
form.addEventListener('submit', (event) => {  
  event.preventDefault(); // Evita que el form recargue la página  
  console.log('Formulario enviado');  
});
```

■ **Tip:** Siempre usa **event.preventDefault()** en formularios para controlar el envío desde JavaScript en lugar de dejar que el navegador recargue la página.

CAPÍTULO 12

¿Y Ahora Qué?

Has terminado la base de JavaScript. Aquí te muestro el camino a seguir.

Temas intermedios que debes aprender

→ **Promesas y async/await**

Manejo de operaciones asíncronas como llamadas a APIs

→ **Fetch API**

Consumir APIs REST para obtener datos de internet

→ **Módulos ES6**

Organizar el código en archivos separados con import/export

→ **Clases y POO**

Programación orientada a objetos en JavaScript moderno

→ **LocalStorage**

Guardar datos en el navegador del usuario

→ **Expresiones regulares**

Buscar y validar patrones en texto

Frameworks populares

Framework	Para qué sirve	Nivel
React	Interfaces de usuario dinámicas	Intermedio
Vue.js	Alternativa más sencilla a React	Principiante+
Angular	Aplicaciones empresariales complejas	Intermedio-Avanzado
Node.js	JavaScript en el servidor (backend)	Intermedio
Next.js	React con renderizado del servidor	Avanzado

Recursos gratuitos recomendados

- MDN Web Docs (developer.mozilla.org) — la documentación oficial más completa
- javascript.info — curso en español, gratuito y muy completo
- freeCodeCamp.org — práctica con proyectos reales certificados
- The Odin Project (theodinproject.com) — ruta completa de desarrollo web
- YouTube: Midudev, Hola Mundo, Fazt Code (canales en español)

CAPÍTULO BONUS

Errores Comunes al Aprender JS

Aprende de los errores más frecuentes para evitarlos desde el principio.

Los 10 errores más frecuentes

1. Usar == en vez de ===

El operador == convierte tipos automáticamente generando resultados inesperados.

```
// MAL:  
if (0 == false) // true – confuso  
  
// BIEN:  
if (0 === false) // false – correcto
```

2. Confundir let, const y var

var tiene alcance de función y no de bloque, lo que genera bugs difíciles de detectar.

```
// var: problema de alcance  
for (var i = 0; i < 3; i++) {}  
console.log(i); // 3 – var escapa del bloque  
  
// let: correcto  
for (let j = 0; j < 3; j++) {}  
// console.log(j); // ReferenceError – bien!
```

3. Mutar arrays/objetos accidentalmente

Los arrays y objetos se pasan por referencia, no por valor.


```
const arr = [1, 2, 3];  
const copia = arr;    // NO es una copia real  
copia.push(4);  
console.log(arr); // [1, 2, 3, 4] – arr también cambió!  
  
// SOLUCIÓN: usar spread  
const copiaReal = [...arr];  
copiaReal.push(5);  
console.log(arr); // [1, 2, 3, 4] – sin cambios
```

4. Olvidar return en funciones

Sin return explícito, la función devuelve undefined.

```
function sumar(a, b) {  
  a + b; // ← falta el return!  
}  
console.log(sumar(2, 3)); // undefined  
  
// BIEN:  
function sumarBien(a, b) { return a + b; }
```

5. this en arrow functions

Las arrow functions no tienen su propio this.

```
const obj = {  
  nombre: 'Carlos',  
  
  // MAL: arrow function pierde 'this'  
  saludarMal: () => console.log('Hola ' + this.nombre),  
  
  // BIEN: función normal conserva 'this'  
  saludarBien() { console.log('Hola ' + this.nombre); }
```

```
};
```

CAPÍTULO BONUS 2

3 Proyectos para Practicar

La mejor forma de aprender es construyendo. Aquí tienes tres proyectos progresivos.

Proyecto 1: Calculadora de IMC

Calcula el Índice de Masa Corporal con los datos del usuario. Perfecto para practicar variables, operaciones y condicionales:

```
function calcularIMC(peso, altura) {  
  // altura en metros, peso en kg  
  const imc = peso / (altura * altura);  
  let categoria;  
  
  if (imc < 18.5) categoria = 'Bajo peso';  
  else if (imc < 25) categoria = 'Normal';  
  else if (imc < 30) categoria = 'Sobrepeso';  
  else categoria = 'Obesidad';  
  
  return {  
    imc: imc.toFixed(2),  
    categoria  
  };  
}  
  
const resultado = calcularIMC(70, 1.75);  
console.log(`IMC: ${resultado.imc} - ${resultado.categoria}`);  
// IMC: 22.86 - Normal
```

Proyecto 2: Lista de Tareas (To-Do List)

Maneja un array de tareas con operaciones CRUD básicas:

```
let tareas = [];  
  
let nextId = 1;  
  
function agregarTarea(texto) {  
  tareas.push({ id: nextId++, texto, completada: false });  
}  
  
function completarTarea(id) {  
  const tarea = tareas.find(t => t.id === id);  
  if (tarea) tarea.completada = true;  
}  
  
function eliminarTarea(id) {  
  tareas = tareas.filter(t => t.id !== id);  
}  
  
function mostrarTareas() {  
  tareas.forEach(t => {  
    const estado = t.completada ? '■' : '□';  
    console.log(`${estado} [${t.id}] ${t.texto}`);  
  });  
}  
  
agregarTarea('Aprender JavaScript');  
agregarTarea('Practicar con proyectos');  
agregarTarea('Leer documentación de MDN');  
completarTarea(1);  
mostrarTareas();
```

Proyecto 3: Consumir una API pública

Usa fetch para obtener datos reales de internet (requiere conocer Promesas):

```
// API pública gratuita: información de países
async function buscarPais(nombre) {
  try {
    const url = `https://restcountries.com/v3.1/name/${nombre}`;
    const respuesta = await fetch(url);

    if (!respuesta.ok) throw new Error('País no encontrado');

    const datos = await respuesta.json();
    const pais = datos[0];

    console.log('País:', pais.name.common);
    console.log('Capital:', pais.capital?.[0]);
    console.log('Población:', pais.population.toLocaleString());
    console.log('Región:', pais.region);

  } catch (error) {
    console.error('Error:', error.message);
  }
}

buscarPais('México');
// País: Mexico
// Capital: Mexico City
// Población: 128,932,753
// Región: Americas
```

■ **Tip:** Intenta completar los 3 proyectos antes de avanzar a frameworks. La base sólida en JavaScript vanilla te hará mucho mejor desarrollador.

Glosario de Términos

API	Interfaz de Programación de Aplicaciones. Permite que dos programas se comuniquen.
Array	Lista ordenada de elementos accesibles por un índice numérico que empieza en 0.
Asíncrono	Operación que no bloquea la ejecución del programa mientras espera un resultado.
Boolean	Tipo de dato con solo dos valores posibles: true o false.
Callback	Función que se pasa como argumento a otra función para ejecutarse después.
Closure	Función que recuerda las variables del entorno donde fue creada.
Consola	Herramienta del navegador para ver mensajes, errores y ejecutar JS.
DOM	Document Object Model. Representación en árbol de los elementos HTML de la página.
ECMAScript	Estándar oficial en el que se basa JavaScript. ES6 (2015) fue una revolución.
Evento	Acción del usuario (clic, tecla, scroll) que el código puede detectar y manejar.
fetch()	Función nativa para hacer peticiones HTTP a servidores o APIs desde el navegador.
Función	Bloque de código reutilizable que realiza una tarea específica y puede retornar un valor.

Hoisting	Comportamiento de JS que mueve las declaraciones al inicio de su ámbito de ejecución.
JSON	JavaScript Object Notation. Formato de texto ligero para intercambiar datos entre sistemas.
Módulo	Archivo JS independiente que exporta funcionalidades para usar en otros archivos.
Null	Valor especial que representa la ausencia intencional de cualquier objeto o valor.
Objeto	Colección de pares clave-valor que representa una entidad con propiedades y métodos.
Parámetro	Variable definida en la declaración de una función para recibir valores externos.
Promise	Objeto que representa el resultado futuro (éxito o fallo) de una operación asíncrona.
Scope	Ámbito de visibilidad de una variable: el contexto en que puede ser accedida.
String	Tipo de dato para representar texto, encerrado entre comillas simples, dobles o backticks.
Template Literal	Cadena con backticks (``) que permite interpolación de variables con \${expresión}.
undefined	Valor que tiene una variable que fue declarada pero a la que no se le asignó valor.
Variable	Espacio en memoria con un nombre simbólico donde se almacena un valor.

Hoja de Referencia Rápida

Guarda esta página como referencia mientras programas.

Variables	<pre>const x = 5; // valor fijo let y = 10; // valor cambiable typeof x; // tipo de dato</pre>
Strings	<pre>'hola'.toUpperCase() // HOLA 'hola mundo'.split(' ') // ['hola','mundo'] str.trim() // quitar espacios str.includes('texto') // boolean `Hola \${nombre}` // template literal</pre>
Numeros	<pre>Math.round(4.6) // 5 Math.floor(4.9) // 4 Math.ceil(4.1) // 5 Math.random() // 0 a 1 Math.max(1,5,3) // 5 num.toFixed(2) // '3.14'</pre>

Arrays	<pre>arr.push(x) // agregar al final arr.pop() // quitar el ultimo arr.map(fn) // transformar todos arr.filter(fn) // filtrar arr.find(fn) // primer coincidente arr.includes(x) // existe x? [...arr] // copiar array</pre>
Objetos	<pre>obj.clave // leer obj.clave = val // asignar delete obj.clave // eliminar Object.keys(obj) // claves Object.values(obj) // valores { ...obj, extra: 1 } // copiar + agregar</pre>
DOM	<pre>document.querySelector('#id') el.textContent = 'texto' el.style.color = 'red' el.classList.add('clase') el.classList.toggle('c') el.addEventListener('click', fn) document.createElement('div') parent.appendChild(hijo)</pre>

Async / Fetch

```
async function getData() {  
  
  const res = await fetch(url);  
  
  const data = await res.json();  
  
  return data;  
  
}
```

¡Felicidades por llegar hasta aquí!

Aprender JavaScript es uno de los pasos más valiosos que puedes dar en tu carrera. La clave está en la práctica diaria: escribe código aunque sean 20 minutos al día.

— Carlos Mendoza